SistemaGestionAcademica
-alumnos: Map<String, Alumno>
-profesores: Map<String, Profesor>
-pilaDeshacer: PilaComandosNota
-colaCertificados: Queue<Alumno>
-gestorArchivos: GestorArchivos
+SistemaGestionAcademica()
+iniciarMenuConsola(): void
+registrarAlumno(cedula: String, nombre: String, correo: String, tipoPrograma: String): void
+registrarProfesor(cedula: String, nombre: String, correo: String, especialidad: String, materia: String): void
+registrarNotaAlumno(cedula: String, nota: Double): Boolean
+deshacerUltimaNota(): Boolean
+generarColaCertificados(): Integer
+mostrarReporteGeneral(): void
+salirYGuardar(): void
Persona
#cedula: String
#nombre: String
#correo: String
+Persona(cedula: String, nombre: String, correo: String)
+getCedula(): String
+getNombre(): String
+getCorreo(): String
+setNombre(nombre: String): void
+setCorreo(correo: String): void
+obtenerInformacion(): String
1
1
realiza I/O
1
1
usa para Ctrl+Z
1
crea instancia
1
gestiona en memoria
*
1
gestiona en memoria
*
GestorArchivos
-rutaAlumnos: String
-rutaProfesores: String
-rutaCertificados: String
+GestorArchivos(rutaAlumnos: String, rutaProfesores: String, rutaCertificados: String)
+escribirAlumnoEnArchivo(alumno: Alumno): Boolean
+escribirProfesorEnArchivo(profesor: Profesor): Boolean
+actualizarTodosLosAlumnos(alumnos: Map<String, Alumno>): Boolean
+cargarAlumnosDesdeArchivo(): Map<String, Alumno>
+cargarProfesoresDesdeArchivo(): Map<String, Profesor>
+exportarCertificadosFIFO(colaGraduandos: Queue<Alumno>): Boolean
PilaComandosNota
-historial: Stack<RegistroNota>
+apilarAccion(registro: RegistroNota): void
+desapilarAccion(): RegistroNota
+estaVacia(): Boolean
Profesor
-especialidad: String
-materia: String
+Profesor(cedula: String, nombre: String, correo: String, especialidad: String, materia: String)
+getEspecialidad(): String
+getMateria(): String
+setEspecialidad(especialidad: String): void
+setMateria(materia: String): void
+obtenerInformacion(): String
Alumno
-programa: ProgramaAcademico
-notas: List<Double>
+Alumno(cedula: String, nombre: String, correo: String, programa: ProgramaAcademico)
+getPrograma(): ProgramaAcademico
+getNotas(): List<Double>
+agregarNota(nota: Double): Boolean
+removerUltimaNota(): Double
+calcularPromedio(): Double
+estaAprobado(): Boolean
+obtenerInformacion(): String
1
almacena LIFO
*
1
cursa
1
RegistroNota
-cedulaAlumno: String
-nota: Double
+RegistroNota(cedulaAlumno: String, nota: Double)
+getCedulaAlumno(): String
+getNota(): Double
ProgramaAcademico
#nombre: String
+ProgramaAcademico(nombre: String)
+getNombre(): String
+evaluarAprobacion(notas: List<Double>): Boolean
Curso
+Curso()
+evaluarAprobacion(notas: List<Double>): Boolean
+(aprueba si promedio >= 10)
Diplomado
+Diplomado()
+evaluarAprobacion(notas: List<Double>): Boolean
+(aprueba si promedio >= 14)
Bootcamp
+Bootcamp()
+evaluarAprobacion(notas: List<Double>): Boolean
+(aprueba si NINGUNA nota < 14)